

# Tag 2

## Datenstrukturen

Listen, Tupel, Dictionaries, Sets & verschachtelte Daten

## Was du heute lernst

1. Wie **Listen** mehrere Werte speichern
2. Was ein **Tupel** ist
3. Wie **Dictionaries** mit Schlüssel und Wert funktionieren
4. Wofür **Sets** verwendet werden
5. Wie `sorted()` Daten sortiert
6. Wie man **verschachtelte Datenstrukturen** liest und verarbeitet

# Block 1

Listen

## Listen durchlaufen

Mit einer `for`-Schleife können wir jeden Wert  
einzeln verarbeiten.

```
for temperatur in temperaturen:  
    print(temperatur)
```

Python nimmt nacheinander  
jeden Wert aus der Liste.

## Eine neue Liste aufbauen

Wir können während einer Schleife  
eine neue Liste erstellen.

```
hohe_werte = []  
  
for temperatur in temperaturen:  
    if temperatur >= 20:  
        hohe_werte.append(temperatur)
```

hohe\_werte enthält danach  
nur Temperaturen ab 20 Grad.

# Block 2

## Tupel

# Was ist ein Tupel?

Ein Tupel speichert ebenfalls mehrere Werte  
in einer festen Reihenfolge.

```
messbereich = (18.4, 23.1)
```

Tupel sehen ähnlich aus wie Listen.

Der Unterschied:

**Ein Tupel wird nicht auf dieselbe Weise verändert wie eine Liste.**

## Auf Tupel zugreifen

Auch Tupel besitzen Indizes.

```
messbereich = (18.4, 23.1)  
  
print(messbereich[0])  
print(messbereich[1])
```

Ausgabe:

```
18.4  
23.1
```

## Wofür sind Tupel praktisch?

Tupel eignen sich für

**zusammengehörige Werte**, die gemeinsam behandelt werden.

Zum Beispiel:

```
messbereich = (18.4, 23.1)
```

Minimum und Maximum  
gehören zusammen.

## Mehrere Werte gemeinsam zurückgeben

Eine Funktion kann mehrere Werte zurückgeben.

```
def min_max(messwerte):  
    kleinster = min(messwerte)  
    groesster = max(messwerte)  
  
    return kleinster, groesster
```

## Das Ergebnis ist ein Tupel

```
ergebnis = min_max(  
    [18.4, 21.7, 16.2, 23.1]  
)  
  
print(ergebnis)
```

Ausgabe:

```
(16.2, 23.1)
```

Mehrere zusammengehörige Werte  
können gemeinsam zurückgegeben werden.

# Liste oder Tupel?

## Liste

```
temperaturen = [18.4, 21.7, 19.2]
```

→ mehrere Werte in einer Reihenfolge

## Tupel

```
messbereich = (18.4, 23.1)
```

→ zusammengehörige Werte

# Block 3

## Dictionaries

# Was ist ein Dictionary?

Ein Dictionary speichert Daten als

**Schlüssel → Wert**

auf Englisch:

**key → value**

```
person = {  
    "name": "Sophie",  
    "alter": 32,  
    "stadt": "Hannover"  
}
```

Ein Dictionary besteht aus

**Schlüsseln ( keys ) und Werten ( values ).**

# Key und Value

Bei:

```
person = {  
    "name": "Sophie",  
    "alter": 32,  
    "stadt": "Hannover"  
}
```

| key     | value      |
|---------|------------|
| "name"  | "Sophie"   |
| "alter" | 32         |
| "stadt" | "Hannover" |

**Key = Schlüssel**

**Value = Wert**

# Ein Key zeigt auf einen Value

Der Key beschreibt,

**welche Information** gespeichert ist.

Der Value enthält

**die eigentliche Information.**

```
person = {  
    "name": "Sophie",  
    "alter": 32  
}
```

Hier bedeutet:

```
"name" → "Sophie"  
"alter" → 32
```

# Auf einen Value zugreifen

Wir verwenden den **Key**:

```
person = {  
    "name": "Sophie",  
    "alter": 32  
}  
  
print(person["name"])  
print(person["alter"])
```

Ausgabe:

```
Sophie  
32
```

Nicht:

```
person[0]
```

Sondern:

```
person["name"]
```

## Keys müssen eindeutig sein

Innerhalb eines Dictionaries kann ein Key nicht sinnvoll mehrfach vorkommen.

```
person = {  
    "name": "Sophie",  
    "alter": 32  
}
```

Die Keys sind:

```
"name"  
"alter"
```

Jeder Key beschreibt eine bestimmte Information.

## Werte können unterschiedlich sein

Ein Dictionary kann verschiedene Datentypen enthalten.

```
server = {  
    "name": "web01",  
    "status": "online",  
    "ram": 32,  
    "backup": True  
}
```

```
"web01"  
"online"  
32  
True
```

Der Key beschreibt die Information.

Der Value enthält die Information.

## Werte verändern

Ein vorhandener Value kann  
über seinen Key verändert werden.

```
server = {  
    "name": "web01",  
    "status": "offline"  
}  
  
server["status"] = "online"
```

Danach:

```
{  
    "name": "web01",  
    "status": "online"  
}
```

## Neue Keys hinzufügen

Ein neuer Key kann einfach  
mit einem Value angelegt werden.

```
server = {  
    "name": "web01",  
    "status": "online"  
}  
  
server["ram"] = 32
```

Danach:

```
{  
    "name": "web01",  
    "status": "online",  
    "ram": 32  
}
```

# Prüfen, ob ein Key vorhanden ist

Mit `in` können wir prüfen,  
ob ein bestimmter Key existiert.

```
server = {  
    "name": "web01",  
    "status": "online"  
}  
  
if "status" in server:  
    print("Status vorhanden")
```

`in` prüft bei einem Dictionary  
standardmäßig die **Keys**.

# Alle Keys

Mit `.keys()` erhalten wir  
alle Schlüssel.

```
server = {  
    "name": "web01",  
    "status": "online",  
    "ram": 32  
}  
  
print(server.keys())
```

Das Ergebnis enthält:

```
name  
status  
ram
```

# Alle Values

Mit `.values()` erhalten wir  
alle Werte.

```
server = {  
    "name": "web01",  
    "status": "online",  
    "ram": 32  
}  
  
print(server.values())
```

Das Ergebnis enthält:

```
web01  
online  
32
```

# Keys durchlaufen

Mit einer `for`-Schleife können wir alle Keys durchlaufen.

```
server = {  
    "name": "web01",  
    "status": "online",  
    "ram": 32  
}  
  
for key in server:  
    print(key)
```

Ausgabe:

```
name  
status  
ram
```

# Key und Value gemeinsam

Oft brauchen wir

Key und Value gleichzeitig.

Dafür gibt es `.items()`.

```
server = {  
    "name": "web01",  
    "status": "online",  
    "ram": 32  
}  
  
for key, value in server.items():  
    print(key, value)
```

## Was passiert bei `items()`?

Python gibt bei jedem Durchlauf

ein Paar aus:

| key      | value    |
|----------|----------|
| "name"   | "web01"  |
| "status" | "online" |
| "ram"    | 32       |

Deshalb können wir schreiben:

```
for key, value in server.items():
```

key → Schlüssel

value → zugehöriger Wert

# Ein Dictionary ausgeben

Mit `.items()` können wir

alle Informationen übersichtlich ausgeben.

```
person = {  
    "name": "Sophie",  
    "alter": 32,  
    "stadt": "Hannover"  
}  
  
for key, value in person.items():  
    print(key, ":", value)
```

Ausgabe:

```
name : Sophie  
alter : 32  
stadt : Hannover
```

# Du bist dran

Gegeben:

```
buch = {  
    "titel": "Python lernen",  
    "seiten": 320,  
    "verfuegbar": True  
}
```

1. Gib den Titel aus.
2. Ändere die Seitenzahl auf 350 .
3. Füge "sprache": "Deutsch" hinzu.
4. Gib alle Keys aus.
5. Gib alle Values aus.
6. Gib mit `.items()` Key und Value gemeinsam aus.

# Dictionaries zum Zählen

Ein Dictionary kann auch  
Häufigkeiten speichern.

```
farben = [  
    "rot",  
    "blau",  
    "rot",  
    "gruen",  
    "blau",  
    "rot"  
]
```

Wir möchten wissen:

Wie oft kommt jede Farbe vor?

# Einen Zähler aufbauen

```
zaehler = {}  
  
for farbe in farben:  
    if farbe in zaehler:  
        zaehler[farbe] = zaehler[farbe] + 1  
    else:  
        zaehler[farbe] = 1
```

```
{  
    "rot": 3,  
    "blau": 2,  
    "gruen": 1  
}
```

Key → Farbe

Value → Anzahl

# Das Grundprinzip

Beim Zählen:

Ist der Key schon vorhanden?

— Ja



Value erhöhen

— Nein



Value = 1

Dieses Muster ist sehr häufig,  
wenn Daten ausgewertet werden.

## Du bist dran

Gegeben:

```
status = [  
    "online",  
    "offline",  
    "online",  
    "wartung",  
    "offline",  
    "online"  
]
```

Erstelle ein Dictionary,  
das zählt, wie oft jeder Status vorkommt.

Erwartetes Ergebnis:

```
{  
  "online": 3,  
  "offline": 2,  
  "wartung": 1  
}
```

# Block 4

## Sets

## Was ist ein Set?

Ein Set speichert

**eindeutige Werte.**

Doppelte Werte werden nur einmal gespeichert.

```
stationen = {  
    "Nord",  
    "Sued",  
    "Nord",  
    "West"  
}
```

# Das Ergebnis

Das Set enthält nur:

```
{  
  "Nord",  
  "Sued",  
  "West"  
}
```

"Nord" war zweimal vorhanden.

Im Set steht der Wert trotzdem  
**nur einmal.**

## Wofür sind Sets praktisch?

Sets eignen sich besonders,  
wenn **Duplikate entfernt** werden sollen.

```
stationen = [  
    "Nord",  
    "Sued",  
    "Nord",  
    "West",  
    "Sued"  
]  
  
eindeutig = set(stationen)
```

Danach:

```
{  
  "Nord",  
  "Sued",  
  "West"  
}
```

# Gemeinsame Werte finden

Sets können miteinander verglichen werden.

Mit `&` erhalten wir die

**Schnittmenge.**

```
sensoren_a = {  
    "temperatur",  
    "druck",  
    "feuchtigkeit"  
}  
  
sensoren_b = {  
    "temperatur",  
    "licht",  
    "feuchtigkeit"  
}
```

# Die Schnittmenge

```
gemeinsam = sensoren_a & sensoren_b
```

Ergebnis:

```
{  
    "temperatur",  
    "feuchtigkeit"  
}
```

Das sind die Werte,  
die in **beiden Sets** vorkommen.

# Liste oder Set?

## Liste

```
werte = [20, 20, 21]
```

- Reihenfolge wichtig
- Duplikate möglich

## Set

```
werte = {20, 21}
```

- eindeutige Werte
- Mengen vergleichen

# Block 5

## Verschachtelte Datenstrukturen

# Datenstrukturen kombinieren

Datenstrukturen können

**miteinander kombiniert** werden.

Zum Beispiel:

Ein Dictionary enthält eine Liste.

```
station = {  
    "name": "Nord",  
    "temperaturen": [18.4, 19.1, 20.3]  
}
```

## Auf die Liste zugreifen

```
station["temperaturen"]
```

Ergebnis:

```
[18.4, 19.1, 20.3]
```

## Auf einen einzelnen Wert zugreifen

Jetzt kombinieren wir

**Dictionary + Liste + Index.**

```
station["temperaturen"][0]
```

Ergebnis:

```
18.4
```

Erst über den Schlüssel zur Liste.

Dann über den Index zum Wert.

## Liste mit Dictionaries

Mehrere Stationen können  
in einer Liste gespeichert werden.

```
stationen = [  
    {  
        "name": "Nord",  
        "temperaturen": [18.0, 20.0]  
    },  
    {  
        "name": "Sued",  
        "temperaturen": [22.0, 24.0]  
    }  
]
```

## Wie ist die Struktur aufgebaut?

```
└─ Liste  
   └─ Dictionary  
      └─ Liste
```

Die äußere Struktur ist eine Liste.

Jedes Element darin ist ein Dictionary.

## Verschachtelte Daten durchlaufen

Wir können die Liste

mit einer Schleife verarbeiten.

```
for station in stationen:  
    name = station["name"]  
    temperaturen = station["temperaturen"]  
  
    print(name)  
    print(temperaturen)
```

## Schritt für Schritt lesen

Bei verschachtelten Daten

nicht alles gleichzeitig lesen.

```
station = stationen[0]
```

Jetzt haben wir das erste Dictionary.

## Den Namen lesen

```
name = station["name"]
```

Jetzt haben wir:

```
Nord
```

## Die Temperaturen lesen

```
temperaturen = station["temperaturen"]
```

Jetzt haben wir:

```
[18.0, 20.0]
```

## Einen einzelnen Messwert lesen

```
erste_temperatur = temperaturen[0]
```

Jetzt haben wir:

```
18.0
```

Schritt für Schritt:

**Liste → Dictionary → Liste → Wert**

# Block 7

Welche Datenstruktur passt?

## Die richtige Struktur wählen

Nicht fragen:

**„Welche Struktur kenne ich?“**

Sondern:

**„Welche Struktur passt zu meinen Daten?“**

# Liste

Verwende eine Liste,  
wenn mehrere Werte  
**in einer Reihenfolge** gespeichert werden sollen.

```
temperaturen = [  
    18.4,  
    19.1,  
    20.3  
]
```

# Tupel

Verwende ein Tupel,  
wenn mehrere zusammengehörige Werte  
**gemeinsam behandelt** werden sollen.

```
messbereich = (  
    18.4,  
    20.3  
)
```

# Dictionary

Verwende ein Dictionary,

wenn Werte über

**Namen oder Schlüssel** angesprochen werden sollen.

```
station = {  
    "name": "Nord",  
    "temperatur": 18.4  
}
```

# Set

Verwende ein Set,  
wenn nur **eindeutige Werte** wichtig sind  
oder Mengen verglichen werden.

```
sensoren = {  
    "temperatur",  
    "druck",  
    "feuchtigkeit"  
}
```

# Verschachtelte Daten

Verwende Kombinationen,  
wenn die Daten selbst  
**eine Struktur besitzen.**

```
stationen = [  
    {  
        "name": "Nord",  
        "temperaturen": [18.0, 20.0]  
    },  
    {  
        "name": "Sued",  
        "temperaturen": [22.0, 24.0]  
    }  
]
```

## Der schnelle Überblick

| Struktur      | Typischer Einsatz            |
|---------------|------------------------------|
| Liste         | mehrere Werte in Reihenfolge |
| Tupel         | zusammengehörige Werte       |
| Dictionary    | Schlüssel → Wert             |
| Set           | eindeutige Werte             |
| Verschachtelt | strukturierte Daten          |

**Was du heute gelernt hast**

# Die wichtigsten Bausteine

## ✓ Liste

Mehrere Werte in einer Reihenfolge

## ✓ Tupel

Zusammengehörige Werte

## ✓ Dictionary

Schlüssel → Wert

## ✓ Set

Eindeutige Werte

# Ausblick auf Tag 3

## Daten dauerhaft speichern

Bisher leben unsere Daten

**direkt im Python-Programm.**

Morgen schauen wir uns Dateien an.

## Morgen geht es um

- Textdateien
- CSV-Dateien
- JSON-Dateien
- Dateien lesen
- Dateien schreiben
- strukturierte Daten aus Dateien verarbeiten

**Von Daten im Programm  
zu Daten auf der Festplatte**